

OAuth 2.0: Technical Documentation

Theory & GRACE Project Implementation

Part 1: What is OAuth 2.0?

OAuth 2.0 is an **authorization framework** (not an authentication protocol) that allows a third-party application to gain **limited access** to a user's account on another service — without ever seeing the user's password.

The Core Problem It Solves

Before OAuth existed, if an app wanted access to your Google Calendar, it would just ask for your Google username and password and store them. This is catastrophically insecure:

- The third-party app now has full control over your account.
- If the app is breached, your Google account is compromised.
- You can't revoke access without changing your password.

OAuth solves this by having Google issue a short-lived **access token** to the third-party app instead. The app never sees your password. You can revoke the token at any time without affecting your Google account.

Part 2: The 4 Roles in OAuth

Role	Description	In GRACE
Resource Owner	The user who owns the data	Prashant
Client	The third-party app requesting access	The GRACE backend (<code>api-worker</code>)
Authorization Server	The server that issues tokens after user consent	Google's OAuth 2.0 server (<code>accounts.google.com</code>)
Resource Server	The API that holds the data the client wants	Google Calendar API

Part 3: Prerequisites — Registering Your App in Google Cloud Console

Before any OAuth flow can happen, Google needs to know your application exists. This is called **App Registration**, and it happens entirely in the Google Cloud Console — one time, before any code runs.

[!IMPORTANT]

This step is done **by the developer** (you), not the user. It produces the `credentials.json` file that GRACE reads on every startup. Without it, the entire OAuth system is dead.

Step-by-Step: How We Obtained `credentials.json`

1. Create a Project in Google Cloud Console

- Go to `console.cloud.google.com`
- Create a new project (e.g., "GRACE")
- This gives Google a namespace to associate all your app's API usage and credentials with

2. Enable the Google Calendar API

- In the left sidebar → APIs & Services → Library
- Search for **Google Calendar API** and click **Enable**
- This tells Google: "My app is allowed to call this specific API"
- If you skip this, even a valid token will get a `403 Access Not Configured` error

3. Configure the OAuth Consent Screen

- Go to APIs & Services → OAuth consent screen
- Choose **External** (any Google account can use it) or **Internal** (only accounts in your Google Workspace org)
- Fill in:
 - **App name:** GRACE
 - **User support email:** your email
 - **Developer contact email:** your email
- This is the consent screen Prashant sees when clicking "Allow" — it shows your app name, logo, and what permissions it's requesting
- Add the scope: `https://www.googleapis.com/auth/calendar`
- Add Prashant's Google account as a **Test User** (required when the app is in "Testing" mode — only test users can authorize it)

4. Create OAuth 2.0 Credentials

- Go to APIs & Services → Credentials → Create Credentials → OAuth client ID
- Choose **Application type: Desktop app** (this is important — it's what produces `redirect_uris: ["http://localhost"]` in the JSON, rather than a web redirect URI)
- Give it a name (e.g., "GRACE Desktop Client")

- Click **Create**

5. Download `credentials.json`

- After creation, Google shows you a dialog with your `client_id` and `client_secret`
- Click **Download JSON** — this is your `credentials.json`
- Place it at: `d:\PERSONAL\GRACE\backend\credentials.json`

What's Inside `credentials.json`

```
{
  "installed": {
    "client_id": "123456789-abcdefg.apps.googleusercontent.com",
    "project_id": "grace-project-id",
    "auth_uri": "https://accounts.google.com/o/oauth2/auth",
    "token_uri": "https://oauth2.googleapis.com/token",
    "auth_provider_x509_cert_url": "https://www.googleapis.com/oauth2/v1/certs",
    "client_secret": "GOCSPX-abc123xyz",
    "redirect_uris": ["http://localhost"]
  }
}
```

Field	What It Is
<code>client_id</code>	Your app's unique public identifier. Safe to share, but should still be protected.
<code>client_secret</code>	Your app's private password. Never expose this. If leaked, rotate it immediately in the Cloud Console.
<code>auth_uri</code>	Where to send the user to start the consent flow (Google's login page)
<code>token_uri</code>	The back-channel endpoint GRACE calls to exchange an auth code for tokens
<code>redirect_uris</code>	The URL Google is allowed to redirect back to after consent. It must exactly match what's in the code.
<code>installed</code>	Indicates this is a Desktop/Native App credential type (vs. <code>web</code> for web apps)

[!NOTE]

The `installed` key is why GRACE uses `credentials.installed` in the code: `const { client_secret, client_id } = require('path').readFileSync('credentials.json', 'utf-8').then(data => { const creds = JSON.parse(data); if (!creds.installed) throw new Error('credentials.json does not contain installed credentials'); return creds; })`. A web-based OAuth app would use `credentials.web` instead.

The Trust Model — Why This Matters

Google's consent screen pulls the app name, logo, and scope description directly from what you configured in the Cloud Console. When Prashant sees:

"GRACE wants to: View and edit events on all your calendars"

...Google is guaranteeing to Prashant that the app claiming to be GRACE has been registered and verified to some degree. The `client_id` in the URL is how Google knows which app is requesting authorization and what consent screen to show.

Part 4: The Authorization Code Flow (How It Works)

GRACE uses the **Authorization Code Grant**, which is the most secure OAuth flow for server-side applications. Here's the complete conceptual flow:

Prashant
(User)

Google's Auth
Server

1. User clicks "Connect Google Calendar"

GRACE Backend generates an Auth URL
and REDIRECTS the browser to Google
2. User sees Google's consent screen:
"Grace wants to access your Google Calendar"
3. User clicks "Allow"
Google generates a ONE-TIME Authorization Code
and redirects back to GRACE's callback URL
4. GRACE backend receives the one-time code
and makes a BACK-CHANNEL request to Google:
"Exchange this code for real tokens"
5. Google responds with:
 - access_token (short-lived, ~1 hour)
 - refresh_token (long-lived, lasts months)
6. GRACE saves these tokens to token.json on disk
and stores the authClient in memory
7. For all future requests, GRACE uses the
access_token. The googleapis library
automatically uses the refresh_token to
get a new access_token when it expires.

Why Two Tokens?

Token	Lifespan	Purpose
Access Token	1 hour	The actual credential used to call the Calendar API
Refresh Token	Months/indefinite	Used in the background to obtain a new access token when the current one expires, without bothering the user again

The access_type: 'offline' parameter in the auth URL is what tells Google to also issue a refresh_token, making the authentication persistent.

Part 5: GRACE's Specific Implementation

Architecture Overview

GRACE Backend (api-worker)

```
credentials.json    ← Your app's identity (Client ID + Secret)
                    NEVER changes. Never put in version control.

token.json          ← Prashant's personal tokens (access + refresh)
                    Written after first login. Auto-refreshed.

src/
  services/
    calendar.service.js ← Core OAuth logic
  routes/
    chat.routes.js      ← HTTP endpoints that expose the OAuth flow
```

Step-by-Step Code Walkthrough

Phase 0: Module Load — `initCalendarAuth()` Auto-Invocation

File: `calendar.service.js`

At the very bottom of `calendar.service.js`, there is a bare function call:

```
// At the very bottom of calendar.service.js:
initCalendarAuth(); // Runs automatically when Node.js first imports this module
```

This is **not** a manually invoked setup script. Node.js executes top-level code in a module the moment it is imported by any other file. Since `llm.service.js` imports from `calendar.service.js` at startup, this line fires automatically every time the server boots.

What it does on first boot (before any OAuth has happened):

1. Reads `credentials.json` → builds an OAuth2 client object with your app's `client_id`, `client_secret`, and `redirect_uri`.
2. Looks for `token.json` on disk.
3. Since `token.json` doesn't exist yet on first boot → logs a warning and **returns** `null`.
4. The module-level `authClient` variable stays `null`.

```
let authClient = null; // Still null after first-boot - no tokens yet
```

At this point the server is running, but calendar integration is inactive. No error is thrown — it just silently waits for the OAuth flow to be triggered.

What it does on subsequent boots (after OAuth is complete):

1. Finds `token.json` on disk → calls `authClient.setCredentials(token)` to load the saved tokens.
2. Sets the global `authClient` in memory.

3. Grace can now call the Calendar API immediately on the first user message.

Phase 1: The Auth URL — GET /api/calendar/auth

Files: chat.routes.js | calendar.service.js

When you want to connect Calendar for the first time, something (the GUI or a direct browser hit) calls:

GET http://localhost:PORT/api/calendar/auth

The route calls `getAuthUrl()`:

```
export function getAuthUrl() {
  // Read app credentials
  const credentials = JSON.parse(fs.readFileSync(CREDENTIALS_PATH, 'utf8'));
  const { client_secret, client_id, redirect_uris } = credentials.installed;

  // Create a TEMPORARY auth client (no tokens yet)
  const authClientTemp = new google.auth.OAuth2(
    client_id, client_secret, redirect_uris[0]
  );

  // Generate the full Google consent page URL
  return authClientTemp.generateAuthUrl({
    access_type: 'offline', // ← Tells Google to give us a refresh_token too
    prompt: 'consent',      // ← Forces the consent screen
    scope: ['https://www.googleapis.com/auth/calendar'], // ← Only Calendar access
  });
}
```

[!NOTE]

A temporary auth client is created here (not the global `authClient`) specifically to generate the URL. The global client may already have tokens attached, and the URL generator expects a clean client.

The route then redirects the user's browser directly to that Google URL:

```
router.get('/calendar/auth', (req, res) => {
  const url = getAuthUrl();
  res.redirect(url); // Browser goes to Google's consent page
});
```

The generated URL looks something like:

```
https://accounts.google.com/o/oauth2/v2/auth
?client_id=YOUR_CLIENT_ID.apps.googleusercontent.com
&redirect_uri=http://localhost:PORT/api/calendar/callback
&response_type=code
&scope=https://www.googleapis.com/auth/calendar
&access_type=offline
&prompt=consent
```

Phase 2: The Callback — Manual Code Extraction + Postman POST

Files: chat.routes.js | calendar.service.js

After clicking "Allow" on Google's consent screen, Google redirects the browser back to the `redirect_uri` registered in `credentials.json`, which for a **Desktop App** credential is:

`http://localhost`

This hits whatever is running on localhost port 80 — in this case, a **bare-bones static IIS website** (Windows' default local web server). There is no GRACE route listening here. The page just renders some minimal HTML or a blank screen. **That's fine** — the important part isn't the page content. It's the URL bar.

The browser's URL now looks like:

`http://localhost/?code=4/OAX4XfWj_abc123...&scope=https://www.googleapis.com...`

The actual setup flow used:

1. Copied the code value from the URL bar manually.
2. Opened **Postman** and sent a POST request directly to the GRACE backend's callback route:

```
POST http://localhost:PORT/api/calendar/callback
Content-Type: application/json
```

```
{
  "code": "4/OAX4XfWj_abc123..."
}
```

The Express route receives this:

```
router.post('/calendar/callback', async (req, res) => {
  const { code } = req.body; // Code extracted from Postman body
  const success = await handleAuthCallback(code);
  if (success) {
    res.json({ success: true, message: "Calendar successfully authenticated!" });
  }
});
```

The `handleAuthCallback(code)` function then does the critical back-channel token exchange:

```
export async function handleAuthCallback(code) {
  const credentials = JSON.parse(fs.readFileSync(CREDENTIALS_PATH, 'utf8'));
  const { client_secret, client_id, redirect_uris } = credentials.installed;
  const client = new google.auth.OAuth2(client_id, client_secret, redirect_uris[0]);

  // *** THE KEY STEP: Exchange the one-time code for real tokens ***
  // This is a back-channel HTTPS POST from the GRACE server to Google.
  // The user's browser is not involved at all in this step.
  const { tokens } = await client.getToken(code);

  client.setCredentials(tokens);

  // Persist tokens to disk - this is what makes auth survive server restarts
  fs.writeFileSync(TOKEN_PATH, JSON.stringify(tokens));

  // Update the global singleton - from this point on, Grace is authenticated
  authClient = client;
  return true;
}
```

```
}
```

[!IMPORTANT]

The code is **one-time use only**. Google invalidates it the moment it's exchanged for tokens. If you try to use the same code twice, you get a `invalid_grant` error. This is a security feature — if someone intercepts the code from the URL, they still can't use it after GRACE has already exchanged it.

The `token.json` written to disk looks like:

```
{
  "access_token": "ya29.a0AfB_...",
  "refresh_token": "1//OgWj...",
  "scope": "https://www.googleapis.com/auth/calendar",
  "token_type": "Bearer",
  "expiry_date": 1750000000000
}
```

Once this file exists on disk, the OAuth setup is complete. This is a **one-time operation** — you will never need to repeat the Postman step unless you delete `token.json` or revoke access from your Google account.

Phase 3: Making Calendar API Calls — The Lazy-Init Guard

File: `calendar.service.js`

Every single exported calendar function starts with the same two-line guard:

```
export async function getCalendarEvents(timeMin, timeMax) {
  // LAZY INIT GUARD - runs every time this function is called
  if (!authClient) await initCalendarAuth();
  if (!authClient) throw new Error("Calendar not authenticated.");
  // ...
}
```

This is called the **lazy initialization** pattern. Here's why it exists and what it does:

Why not just trust the startup call? The `initCalendarAuth()` at module load handles the happy path (`token.json` exists). But what if:

- The server just restarted and the startup call hasn't fully completed yet?
- Some code path calls a calendar function before the module-level call ran?
- `authClient` got set to null due to a bug?

The guard is a safety net. Every calendar method defensively checks `if (!authClient)` and re-runs `initCalendarAuth()` if needed. Since `initCalendarAuth()` itself has an early return (`if (authClient) return authClient;`), calling it multiple times is completely safe — it's idempotent.

The two-line pattern explained:

```
if (!authClient) await initCalendarAuth(); // Attempt to load token.json
if (!authClient) throw new Error("...");   // Still null? Fail loudly
```

Line 1 tries to recover. Line 2 enforces the failure if recovery wasn't possible (meaning OAuth was never set up).

The `googleapis` library handles token refresh **automatically**. When the `access_token` expires (after 1 hour), the library silently uses the `refresh_token` stored in `token.json` to request a fresh one from Google — without any user interaction or code on our side.

Phase 4: Grace Triggering Calendar Tools

File: `llm.service.js`

The Calendar API functions are exposed to Grace as Gemini tool declarations. When you say *"What's on my schedule tomorrow?"*, Gemini doesn't call the API directly. Instead, it returns a structured `functionCall` object. The JavaScript code in `llm.service.js` intercepts this and dispatches it to the right service:

```
// Grace decides to call getCalendarEvents
else if (call.name === "getCalendarEvents") {
  const args = call.args;
  const events = await getCalendarEvents(args.timeMin, args.timeMax);
  toolResult = { success: true, events: events };

  // Also package the data for the GUI hologram display
  calendarData = { events: events, timeMin: args.timeMin, timeMax: args.timeMax };
}
```

The `calendarData` is then passed back in the HTTP response to the Python pipeline (`core/pipeline.py`), which emits a `sig_calendar_update` signal to the PyQt GUI, triggering the `HoloCalendarWidget` popup.

Complete Request Lifecycle (One-Time Setup — The Actual Flow)

0. [Server Starts]
 - Node.js imports `calendar.service.js` → `initCalendarAuth()` auto-runs
 - No `token.json` found → `authClient` stays null → silent warning logged
1. [Browser] Hit: `GET http://localhost:PORT/api/calendar/auth`
2. [Backend: `GET /calendar/auth`]
 - `getAuthUrl()` creates a temp `OAuth2` client
 - Generates full Google consent page URL
 - `res.redirect(url)` → browser is sent to Google
3. [Browser] Lands on Google's consent screen:
 - "GRACE wants to: View and edit events on all your calendars"
4. [User] Clicks "Allow"
5. [Google] Redirects browser to the registered `redirect_uri`:
 - `http://localhost/?code=4/OAX4...&scope=...`
 - (This hits the local IIS static site - a bare bones HTML page)
 - The page content doesn't matter. Only the URL bar matters.
6. [User] Manually copies the "code" value from the browser URL bar
7. [User] Opens Postman and sends:

```
POST http://localhost:PORT/api/calendar/callback
Body: { "code": "4/OAX4..." }
```

8. [Backend: POST /calendar/callback]
 handleAuthCallback(code) is called
 Makes a back-channel HTTPS request to Google's token_uri:
 → Google exchanges the one-time code for:
 - access_token (short-lived, ~1 hour)
 - refresh_token (long-lived)
 Tokens are saved to token.json on disk
 Global authClient is set in memory
9. Setup complete. token.json now exists on disk.
 This step never needs to be repeated.

Complete Request Lifecycle (Day-to-Day Usage)

1. [Server Starts]
 initCalendarAuth() auto-runs → finds token.json → loads tokens
 2. [User] "Grace, what's on my calendar today?"
 3. [llm.service.js] Sends message to Gemini with all tool definitions
 4. [Gemini] Returns a functionCall: getCalendarEvents(timeMin, timeMax)
 5. [llm.service.js] Catches the functionCall, calls getCalendarEvents()
 6. [calendar.service.js: getCalendarEvents()]
 Lazy-init guard runs: authClient is set → guard passes immediately
 Calls Google Calendar API with authClient
 If access_token is expired → googleapis silently fetches a new one
 Returns list of events
 7. [llm.service.js] Packages events as a functionResponse → sends back to Gemini
 8. [Gemini] Reads the events → generates a natural language response
 9. [pipeline.py] Receives calendarData in the API response
 Emits sig_calendar_update signal to the PyQt GUI
 10. [HUD] HoloCalendarWidget popup appears with your events
-

Part 6: Key Files Reference

File	Role
calendar.service.js	Core OAuth2 logic: auth init, URL generation, token exchange, Calendar API calls
chat.routes.js	HTTP endpoints: GET /calendar/auth and POST /calendar/callback
llm.service.js	Gemini tool declarations for calendar functions + dispatch logic
pipeline.py	Receives calendarData from API response, emits GUI signal
credentials.json	App credentials (Client ID + Secret) from Google Cloud Console. Never commit.
token.json	User's access + refresh tokens. Written after first login. Never commit.

Part 7: Token Lifetime & Refresh Mechanics

Access Token — 1 Hour Exactly

Google access tokens are valid for exactly **3,600 seconds (1 hour)** from the moment they are issued. The `expiry_date` field in `token.json` reflects this:

```
{
  "access_token": "ya29.a0AfB...",
  "expiry_date": 17500000000000 ← Unix timestamp in ms (issued_at + 3,600,000ms)
}
```

Once this timestamp is in the past, the access token is rejected by Google's API with a 401 Unauthorized response.

What Happens When the Access Token Expires

The `googleapis` library manages this **entirely automatically** — you write zero code for it. Here is the exact mechanism:

Before making any Calendar API call, the library checks the `expiry_date` of the `access_token` currently loaded in `authClient`. If it has expired (or is within a small safety buffer), the library intercepts and does the following transparently:

1. Makes a back-channel token refresh request to Google:

```
POST https://oauth2.googleapis.com/token
Content-Type: application/x-www-form-urlencoded
```

```
grant_type=refresh_token
&refresh_token=<stored refresh_token from token.json>
&client_id=<your client_id>
&client_secret=<your client_secret>
```

2. Google responds with a fresh access token:

```
{
  "access_token": "ya29.NEW_TOKEN...",
  "expires_in": 3600,
  "token_type": "Bearer"
}
```

3. The library updates `authClient` in memory silently, then proceeds with the original API call.

Your calendar functions — `getCalendarEvents()`, `scheduleEvent()` etc. — **never see any of this**. From their perspective, the call just works. There is no error, no retry, no notification.

[!NOTE]

This refresh does **not** automatically update `token.json` on disk. The new access token lives only in memory in `authClient`. If the server restarts, `token.json` is loaded again, the old (expired) access token is read from it, and the refresh happens again on the first API call. This is fine — the `refresh_token` in `token.json` never expires under normal conditions, so the cycle repeats correctly on every boot.

What Happens at Server Startup (Token Already Expired)

This is an important edge case. When `initCalendarAuth()` runs at startup and loads `token.json`:

```
authClient.setCredentials(token); // Just loads data into memory - NO network call
```

No network call is made here. The library does not proactively refresh the token at startup, even if the `expiry_date` is already in the past. The refresh is lazy — it only triggers the **first time an actual Calendar API call is made**.

So the startup sequence looks like this:

Server starts

- `initCalendarAuth()` loads expired `token.json` into memory (no error)
- `authClient` is set, but `access_token` is stale

First calendar API call arrives (e.g. `getCalendarEvents`)

- `googleapis` checks `expiry_date` → it's in the past
- `googleapis` silently POSTs to `/token` with the `refresh_token`
- Google returns a fresh `access_token`
- API call proceeds with the new token
- (Takes slightly longer this one time due to the extra refresh round-trip)

Refresh Token Lifetime

Unlike access tokens, the `refresh_token` does **not** have a fixed expiry. It is designed to last indefinitely. However, it can become invalid under these specific conditions:

Condition	Effect
Prashant revokes access from Google Account Settings	Refresh token is immediately invalidated
<code>credentials.json</code> is deleted or OAuth Client is deleted	Refresh token is immediately invalidated
App is in Testing mode and not published for 6 months	Google auto-expires all tokens for the app
App has not been used by this user for 6 months	Google may expire the refresh token

[!WARNING]

GRACE uses a **Desktop App credential in Testing mode** in a personal GCP project. The 6-month testing mode expiry is the most relevant risk. If GRACE isn't used for

6 months or the GCP project is inactive, the `refresh_token` in `token.json` will stop working and you'll need to re-run the full Postman OAuth flow to get a new one.

When a refresh token is expired or invalid, the `googleapis` library throws an error like:

Error: `invalid_grant` - Token has been expired or revoked

If you see this, delete `token.json` and repeat the one-time setup flow from Phase 1.

Part 8: Security Notes

[!CAUTION]

Both `credentials.json` and `token.json` must be in `.gitignore`. `credentials.json` is your app's identity — if leaked, anyone can impersonate your app. `token.json` contains Prashant's live access to his Google Calendar — if leaked, anyone can read and modify his calendar.

[!WARNING]

The prompt: `'consent'` flag forces Google to show the consent screen every time `getAuthUrl()` is called, even if the user has already granted access. This is intentional — it ensures a fresh `refresh_token` is issued every time (Google only gives refresh tokens when the consent screen is shown). Without this, re-running the auth flow would only give an `access_token` with no `refresh_token`, breaking the persistent auth.

[!TIP]

The scope is deliberately minimal — `https://www.googleapis.com/auth/calendar`. This means the token can only access Calendar. It cannot read Gmail, Drive, Contacts, etc. Always use the narrowest scope possible (Principle of Least Privilege).